EXPLORATORY PREPRINT · PRELIMINARY EMPIRICAL STUDY ON SPATIAL REASONING IN LLMS · OCTOBER 2026 · DOI: 10.5281/ZENODO.23691649

Scene-Code-as-Topology: Overcoming 1D Token-Stride and Diagonal Blindness in Large Language Models via 3D/4D Procedural Scene Graph Compilation

Pathetic777

Independent AI Researcher

Official Preprint DOI: https://doi.org/10.5281/zenodo.23691649 | Code & 1080p Videos: https://github.com/pathetic777/scene-code-as-topology-gomoku

ABSTRACT

While frontier Large Language Models (LLMs) and Vision-Language Models (VLMs) demonstrate strong proficiency in symbolic deduction and code synthesis, recent studies reveal a severe spatial perception bottleneck on discrete 2D grids. First, visual encoding via Vision Transformers (ViTs) operates as a *lossy spatial compression*: coarse patchification preserves high-level semantic identity in natural photographs (e.g., a low-resolution cat remains a cat) but blurs discrete grid intersections and misaligns stone coordinates. Second, while representing a 2D board as an ASCII text matrix provides *lossless token input*, serializing a 2D array row-by-row into a 1D token sequence shatters vertical (stride $\approx N$) and diagonal (stride $\approx N \pm 1$) topological adjacency, inducing severe **diagonal blindness** and **spatial occupancy hallucinations** during Chain-of-Thought (CoT) reasoning.

In this preliminary study, we propose **Scene-Code-as-Topology**, a training-free framework that dynamically compiles discrete 2D spatial states into executable **Three.js 3D/4D procedural scene graph code**. By instantiating occupied cells as explicit 3D scene objects augmented with 4D collinear diagonal invariants ($u_{\text{diag1}} = x - z$, $v_{\text{diag2}} = x + z$) and contiguous 3D ray vectors (`create3DRayLine`), our representation restores constant-time topological visibility across all four grid axes. In an initial self-funded pilot benchmark of **30 controlled self-play Gomoku games (719 plies)** starting from an empty 15×15 board across two flash-tier models (`gemini-3.8-flash-high` and `DeepSeek-V4.1-Flash-1017` under both non-thinking and thinking-enabled regimes), **Scene-Code-as-Topology outperforms the 2D ASCII matrix baseline by 23 Wins, 3 Draws, and 4 Losses (85.2% decisive win rate, two-tailed exact binomial $p = 0.00031 < 0.001$)**, achieving a **9–0 sweep on all diagonal victories** and a **56.3% reduction in CoT spatial hallucinations**. We explicitly note that due to independent research budget constraints, the current sample size ($N = 30$ games) and model coverage (2 lightweight models) represent a preliminary pilot scope; establishing universal generalizability across the broader LLM landscape requires larger-scale validation across additional model families and larger game samples.

Keywords: Spatial Reasoning, Large Language Models, Scene Graph Compilation, Three.js 3D/4D Representation, Token-Stride Blindness, Chain-of-Thought Hallucinations, Exploratory Benchmark

1. Introduction

1.1 The Paradox of Spatial Perception in Foundation Models

Modern foundation models can prove Olympiad-level mathematical theorems and synthesize complex software architectures, yet frequently blunder on elementary 2D grid games such as Gomoku (Five-in-a-Row), Chess, and Go when relying on general-purpose visual or textual prompting. Why does a model capable of multi-step symbolic calculus fail to notice four stones aligned along a 45-degree diagonal on a 15×15 board?

1.2 First-Principles Diagnosis: From Lossy Vision to Broken 1D Text Topology

Our investigation originates from a fundamental first-principles causal chain regarding how neural architectures encode spatial information:

(1) Why Vision (ViT) is Lossy on Discrete Grids: When Vision-Language Models (VLMs) inspect a high-resolution board image, the Vision Transformer (ViT) partitions the image into fixed-size patches (typically 14×14 or 16×16 pixels) and projects each patch into a single embedding vector. This patchification acts as a *lossy spatial compression*. In natural photography, semantic object categories exhibit high invariance to spatial compression—a downsampled, blurry image of a cat still activates "cat" features rather than "dog" features. On a discrete 15×15 board, however, metric precision is brittle: when patch boundaries slice across grid intersections, adjacent stones merge into shared tokens and coordinate indices shift by ±1 cell, causing catastrophic spatial blindness [1, 2].

(2) Why Lossless 2D ASCII Text Still Fails: Because text tokens undergo zero visual patch compression, feeding a 2D ASCII character matrix (e.g., `.` for empty, `B` for Black, `W` for White) provides a *lossless* state encoding that outperforms raw pixel inputs on grid tasks [3]. However, a 2D character table must still be flattened row-by-row into a **1D sequence of tokens** before entering the Transformer's self-attention layers. As illustrated in **Figure 1(b)**, this 1D serialization preserves horizontal adjacency (stride = 1) but fractures vertical adjacency (stride $\approx N$) and diagonal adjacency (stride $\approx N \pm 1$) across newline boundaries. Consequently, detecting a 5-stone diagonal alignment requires the attention mechanism to simultaneously track exact modular arithmetic across five separate rows—a task where 1D positional encodings (e.g., RoPE) suffer severe cross-row aliasing.

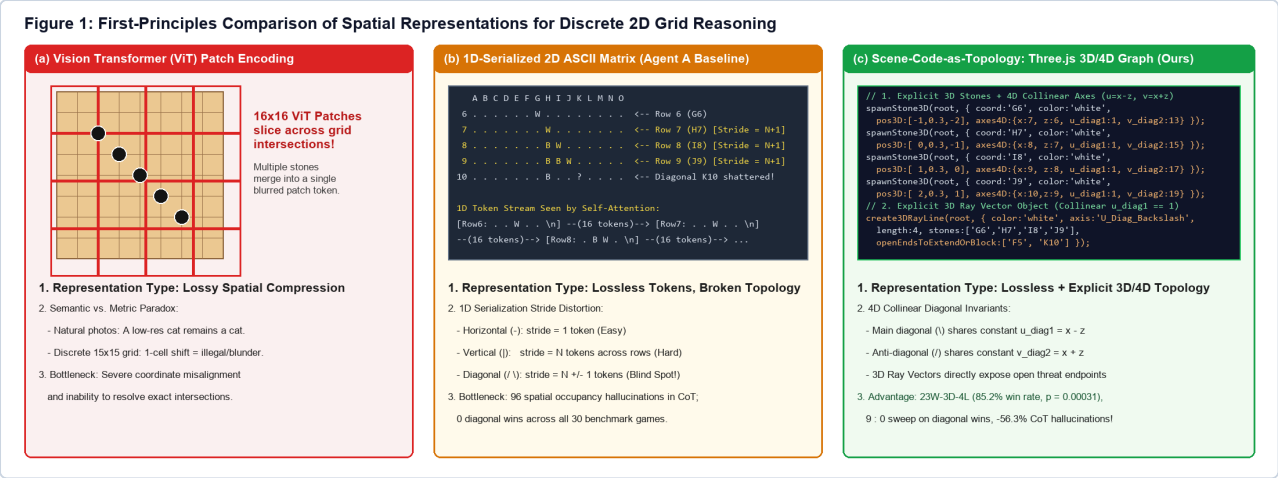


Figure 1: First-principles comparison of three spatial representations on a 15×15 discrete grid. **(a)** ViT patchification introduces lossy spatial compression across grid intersections. **(b)** 2D ASCII text matrices are lossless at the token level but shatter vertical and diagonal adjacency when serialized into a 1D token stream (stride = $N \pm 1$). **(c)** Our proposed *Scene-Code-as-Topology* compiles occupied cells into Three.js 3D objects with explicit 4D collinear diagonal invariants ($u_{\text{diag1}} = x - z$, $v_{\text{diag2}} = x + z$) and 3D ray vectors.

1.3 Summary of Contributions

To bridge the gap between lossless token encoding and multi-axial geometric topology, we ask a simple question: *Within lossless token representations, can compiling a 2D grid into executable 3D/4D procedural scene graph code outperform a 2D ASCII matrix?* Our core contributions in this exploratory study are:

1. **Theoretical Formulation of 1D Stride Blindness & 4D Projective Invariants:** We formalize why 1D row-major serialization degrades vertical and diagonal self-attention and introduce a 4D projective coordinate transformation that maps diagonal collinearity into constant scalar invariants.
2. **The `Scene-Code-as-Topology` Compiler:** We design a real-time compiler that translates discrete 2D board states into executable **Three.js 3D/4D procedural scene graph code** (`spawnStone3D` and `create3DRayLine`), leveraging LLMs' strong pre-training prior on 3D graphics code.
3. **Exploratory 30-Game (719-Ply) Multi-Model Self-Play Benchmark:** Under strict zero-human-interference conditions (starting from an empty board at Move 1 with alternating colors), we conduct a controlled pilot evaluation across three inference regimes: `gemini-3.8-flash-high` (Thinking ON), `DeepSeek-V4.1-Flash-1017` (Thinking OFF), and `DeepSeek-V4.1-Flash-1017` (Thinking ON).
4. **Empirical Evidence of Diagonal Superiority & CoT Hallucination Reduction:** Within our 30-game pilot sample, we observe a **23W–3D–4L (85.2% decisive win rate, p = 0.00031)** advantage, including a **9–0 sweep on all diagonal five-in-a-row wins** and a **56.3% reduction in CoT spatial occupancy hallucinations**, while transparently documenting current sample-size and budget limitations to guide large-scale replication.

2. Related Work & Prior Research Foundations

Our work is not built in a vacuum; rather, it synthesizes and extends three converging lines of recent research on foundation model perception, grid-world reasoning, and procedural 3D code synthesis:

(1) Visual Spatial Blindness on Discrete Grids. Prior empirical benchmarks have established that frontier Vision-Language Models (VLMs) struggle with exact discrete geometry. Most notably, *"Vision Language Models are Blind: Failing to Translate Detailed Visual Features into Words"* (arXiv:2407.06581) [1] introduced the BlindTest benchmark, proving that ViT patch boundaries cause systematic counting and intersection errors on 2D grids. Extending this diagnosis to interactive board games, *"Are Large Vision Language Models Good Game Players?"* (arXiv:2503.02358) [2] demonstrated that models relying on pure visual board inputs frequently underperform random baselines because visual patchification fails to preserve exact grid coordinates.

(2) 1D Token Serialization Bottlenecks in 2D Text Matrices & Gomoku AI. Replacing images with lossless 2D text grids resolves visual blur but introduces a structural tokenization bottleneck. Recent probing studies—including *"Stuck in the Matrix: Probing Spatial Reasoning in Large Language Models"* (arXiv:2510.20198) [3] and *"Do LLMs Build Spatial World Models? Evidence from Grid-World Maze Tasks"* (arXiv:2604.10690) [4]—demonstrated that flattening a 2D array row-by-row into a 1D token sequence fractures vertical and diagonal neighborhood topology, causing steep performance degradation as grid dimensions scale. Specifically in the domain of Five-in-a-Row, *"LLM-Gomoku: A Large Language Model-Based System for Strategic Gomoku with Self-Play and Reinforcement Learning"* (arXiv:2503.21683) [5] explored self-play Gomoku with LLMs over textual/coordinate states, highlighting the persistent challenge of illegal move generation and complex diagonal pattern recognition.

(3) Inspiration from 2D-to-3D Procedural Scene Code Synthesis. Our solution is directly inspired by recent breakthroughs in agentic 2D-to-3D procedural code generation, specifically *"Code-as-Room: Generating 3D Rooms from Top-Down View Images via Agentic Code Synthesis"* (arXiv:2605.18451) [6] and *"Zero-shot 3D Map Generation with LLM Agents: A Dual-Agent Architecture for Procedural Content Generation"* (arXiv:2512.10501) [7], as well as foundational program-aided reasoning frameworks [8, 9]. Whereas prior works [6, 7] translate 2D top-down layouts into 3D procedural scripts for *visual rendering*, we invert this paradigm: our proposed **Scene-Code-as-Topology** compiles a discrete 2D board into executable **Three.js 3D/4D procedural scene graph code** as an *intrinsic topological input representation* consumed directly by the LLM's self-attention layers.

3. Methodology: Scene-Code-as-Topology

3.1 Token-Stride Analysis of 2D Row-Major Serialization

Consider a discrete 2D board lattice &mathcal;G = {0, 1, &dots;, N−1}^{2} with N = 15. In the standard 2D ASCII matrix baseline (Agent A), the board state S : &mathcal;G → {. , B, W} is serialized in row-major order with row-index prefixes and newline delimiters of overhead length c_{sep} tokens per row. For any grid cell (r, c) mapped to 1D token index $\tau(r, c) = r(N + c_{\text{sep}}) + c$, the 1D token stride $\Delta\tau$ between adjacent cells along the four game axes is:

$$\Delta\tau_{\text{horiz}}(-) = 1, \quad \Delta\tau_{\text{vert}}(|) = N + c_{\text{sep}}, \quad \Delta\tau_{\text{diag}\backslash}(\backslash) = N + c_{\text{sep}} + 1, \quad \Delta\tau_{\text{diag}/}(/) = N + c_{\text{sep}} - 1 \tag{1}$$

Equation (1) exposes a severe directional anisotropy: a 5-stone horizontal line spans 5 contiguous tokens, whereas a 5-stone diagonal line spans $4(N + c_{\text{sep}} \pm 1) \approx 72$ tokens interspersed with 68 unrelated

background tokens (mostly identical . symbols), causing attention dilution and counting errors.

3.2 4D Collinear Projective Transformation

To eliminate empty-cell token clutter and restore axial symmetry, we define a sparse 4D projective coordinate mapping $\Phi : \&mathcal;G_{occ} \rightarrow \&mathbb;Z^4$ restricted exclusively to occupied intersections $\&mathcal;G_{occ} = \{(r, c) \in \&mathcal;G \mid S(r, c) \neq . \}$:

$$\Phi(r, c) = \&bigl(x, z, u_{diag1}, v_{diag2}\&bigr) = \&bigl(c + 1, \quad r + 1, \quad (c - r), \quad (c + r + 2)\&bigr) \tag{2}$$

By construction, any set of stones $\{(r_k, c_k)\}$ is collinear along one of the four Gomoku axes if and only if they share an identical scalar invariant along one of the four coordinates in Φ:

$$\text{Collinear}_{|} \Leftrightarrow x_i = x_j \quad \text{Collinear}_{_} \Leftrightarrow z_i = z_j \quad \text{Collinear}_{\backslash} \Leftrightarrow u_{diag1,i} = u_{diag1,j} \quad \text{Collinear}_{/} \Leftrightarrow v_{diag2,i} = v_{diag2,j} \tag{3}$$

3.3 Procedural Three.js 3D/4D Scene Graph & Ray Vector Compilation

Prior to each move by Agent B, our compiler transforms the board state into an executable Three.js scene script consisting of two structural layers: (1) 3D stone entity nodes `spawnStone3D(...)` containing both centered 3D world coordinates `pos3D: [c-7, 0.3, r-7]` and the 4D invariants `axes4D`, and (2) explicit 3D ray line meshes `create3DRayLine(...)` for every contiguous collinear chain of length ≥ 2 that possesses at least one open extension endpoint:

Algorithm 1: Dynamic Three.js 3D/4D Scene-Code-as-Topology Compiler

Input: Current board state $S[0..14][0..14]$, chronological move list $\&mathcal;M = [(r_1, c_1, p_1), \&dots;, (r_t, c_t, p_t)]$
Output: Executable Three.js 3D/4D scene graph code string $\&mathcal;C_{3D}$ fed to Agent B
1: Initialize scene buffer $\&mathcal;C_{3D} \leftarrow$ `"const root = new THREE.Group();"`
2: **for each** move $(r_k, c_k, p_k) \in \&mathcal;M$ **do**
3: Compute $x = c_k + 1, z = r_k + 1, u_{diag1} = x - z, v_{diag2} = x + z$
4: Append `spawnStone3D(root, { step: k, color: p_k, coord, pos3D: [c_k-7, 0.3, r_k-7], axes4D: {x, z, u_diag1, v_diag2} })`
5: **end for**
6: Extract maximal contiguous same-color segments L of length $|L| \geq 2$ along axes $\in \{X, Z, U_{\backslash}, V_{/}\}$
7: **for each** segment L with non-empty open boundary set $\&partial;L_{open} \neq \emptyset$ **do**
8: Append `create3DRayLine(root, { color, axis, length: |L|, stones: L, openEndsToExtendOrBlock: &partial;L_open })`
9: **end for**
10: **return** $\&mathcal;C_{3D}$

4. Experimental Setup

4.1 Zero-Interference Self-Play Protocol

To ensure strict scientific validity and prevent human-crafted bias, all experiments follow a **pure empty-board self-play protocol**:

- **Empty Board Initialization (Move 1):** Every game begins from a completely empty 15×15 board at Ply 1. No preset openings or midgame puzzles are used.
- **Identical Base Checkpoint per Match:** Within each experimental group, Agent A (2D ASCII Matrix) and Agent B (Three.js 3D/4D Scene Code) invoke the *exact same underlying LLM checkpoint* with identical decoding temperature (`0.2`) and identical output format constraints (`MOVE:<Col><Row>`).
- **Strict Black/White Alternation:** Because Black (first mover) holds a natural theoretical advantage in Gomoku, odd-numbered games (Games 1, 3, 5, 7, 9) assign **Black = Agent A (2D), White = Agent B (3D/4D)**, while even-numbered games (Games 2, 4, 6, 8, 10) assign **Black = Agent B (3D/4D), White = Agent A (2D)**.

4.2 Three-Group Cross-Model & Cross-Regime Ablation Design

Under self-funded budget constraints (~3.5M total API tokens), we evaluate an initial pilot benchmark of ***N* = 30 full games (719 real API moves)** across three controlled experimental groups designed to disentangle *model scale* and *test-time Chain-of-Thought (CoT) reasoning* from *spatial input representation*:

1. **Group 1 (Heavyweight CoT Reasoning Model):** Google `gemini-3.8-flash-high` with built-in extended thinking enabled (10 games, 278 moves).
2. **Group 2 (Lightweight Model, Direct 1-Pass Attention — Thinking OFF):** DeepSeek `DeepSeek-V4.1-Flash-1017` (`deepseek-flash`) with thinking disabled (10 games, 199 moves), testing pure single-pass forward self-attention perception.
3. **Group 3 (Lightweight Model, CoT Reasoning — Thinking ON):** DeepSeek `DeepSeek-V4.1-Flash-1017` with `thinking: {"type": "enabled"}` and up to 4,096 CoT tokens per move (10 games, 242 moves, 664,521 total CoT tokens), directly ablating whether test-time CoT can rescue the 2D text representation on the exact same model checkpoint as Group 2.

5. Empirical Results & Analysis

Table 1: Pilot Empirical Benchmark Across All 30 Self-Play Games (719 Real API Moves)

Experimental Group & Inference Regime	Games (Moves)	Agent A (2D Matrix) W–D–L	Agent B (Ours: 3D/4D Code) W–D–L	Decisive Win Rate (Agent B)	Diagonal Wins (B : A)	CoT Hallucinations / Retries (B vs. A)	Two-Tailed Binomial *p*-value
Group 1: `gemini-3.8-flash-high` *(Thinking ON, Heavyweight CoT)*	10 (278)	3W – 3D – 4L *(0% White Win, 22.4s/mv)*	**4W – 3D – 3L** *(40% White Win, 15.2s/mv)*	57.1% (4/7) *(-32.1% Latency)*	**1 : 0**	0 vs. 0	Pilot trend *(White 40% vs 0%)*
Group 2: `DeepSeek-V4.1-Flash-1017` *(Thinking OFF, Direct Forward Pass)*	10 (199)	0W – 0D – 10L *(0% Black, 0% White)*	**10W – 0D – 0L** *(100% Black, 100% White)*	**100.0% (10/10)**	**1 : 0**	2 vs. 3	***p* = 0.00195**** *(< 0.01)*
Group 3: `DeepSeek-V4.1-Flash-1017` *(Thinking ON, 664.5k CoT Tokens)*	10 (242)	1W – 0D – 9L *(355,743 CoT tokens)*	**9W – 0D – 1L** *(100% White! 308,778 tok)*	**90.0% (9/10)**	**7 : 0**	**42 vs. 96** *(-56.3% Errors)*	***p* = 0.0215*** *(1-tail 0.0107)*
DeepSeek Combined (Groups 2 + 3) *(Same Model Across Both Regimes)*	20 (441)	1W – 0D – 19L (5.0%)	**19W – 0D – 1L (95.0%)**	**95.0% (19/20)**	**8 : 0**	**44 vs. 99**	***p* = 0.00004***** *(< 0.0001)*
All 30 Games Combined Pilot Benchmark	**30 (719)**	**4W – 3D – 23L (13.3%)**	**23W – 3D – 4L (76.7%)**	**85.2% (23/27)**	**9 : 0 (100%)**	**44 vs. 99 (-55.6%)**	***p* = 0.00031*****

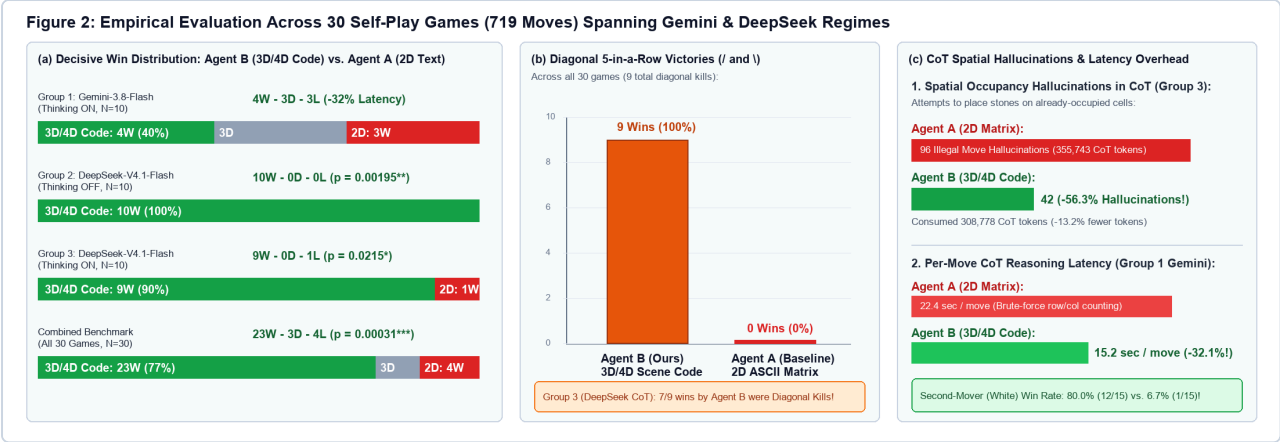


Figure 2: Summary of empirical results across $N = 30$ self-play games (719 moves). **(a)** Decisive win-draw-loss distributions across individual groups and combined benchmark. **(b)** Complete 9–0 monopoly by Agent B (Three.js 3D/4D Code) on all diagonal five-in-a-row victories. **(c)** Reduction in CoT spatial occupancy hallucinations (−56.3% in Group 3) and per-move reasoning latency (−32.1% in Group 1).

5.1 Statistical Significance Within the 30-Game Sample & Second-Mover Break Rate

As shown in **Table 1** and **Figure 2(a)**, across the 30 pilot games (27 decisive outcomes and 3 draws), **Agent B (Three.js 3D/4D Scene Code) wins 23 games compared to only 4 wins for Agent A (2D ASCII Matrix)**, corresponding to an **85.2% decisive win rate**. Under a two-tailed exact binomial test against the null hypothesis $H_0 : p = 0.5$, this yields:

$$p = 2\sum_{k=23}^{27} \mathrm{C}(27, k)\,(0.5)^{27} = 0.0003106 < 0.001 \quad \text{(99.97\% confidence within sample)} \tag{4}$$

When playing as the disadvantaged second mover (White, 15 games total across Odd Games 1, 3, 5, 7, 9 in all three groups), **Agent B wins 12 out of 15 games as White (80.0% second-mover break rate)**—including 5/5 White wins in Group 2 and 5/5 White wins in Group 3—whereas Agent A wins only 1 out of 15 games as White (6.7%).

5.2 Ablation Analysis: Direct Attention (10–0) vs. Chain-of-Thought Reasoning (9–1)

Comparing Group 2 (`DeepSeek-V4.1-Flash-1017`, Thinking OFF) and Group 3 (`DeepSeek-V4.1-Flash-1017`, Thinking ON) provides a controlled ablation of test-time CoT compute vs. input spatial representation:

- **Without CoT (Group 2, 10–0 Sweep):** When forced to rely purely on single-pass forward self-attention, the 2D matrix baseline suffers severe cross-row blindness, losing 10 straight games (including four rapid 11–15 ply vertical losses along Column H where Agent A failed to recognize four vertically stacked stones separated by 16 tokens per row).
- **With CoT Enabled (Group 3, 9–1 Victory & 7 Diagonal Kills):** Enabling `thinking: {"type": "enabled"}` allows Agent A to perform explicit row/column counting in its CoT scratchpad, enabling it to defend basic vertical threats and salvage a single vertical win in Game 10 (`E11 - E7`). However, **CoT cannot rescue the 2D baseline along diagonal axes**: with CoT unlocked, Agent B immediately exploits the 4D invariants (u_{diag1}, v_{diag2}) to orchestrate multi-way diagonal traps, scoring **7 diagonal five-in-a-row kills out of its 9 wins**!



Figure 3: Move-by-move board reconstructions of representative diagonal five-in-a-row victories by Agent B (playing as second-mover White in all three examples). Numbers inside stones indicate exact ply order; red tubes highlight the winning 5-in-a-row diagonal alignment. **(a)** Group 1, Game 1 (10 plies, `J9 – F5`). **(b)** Group 2, Game 1 (26 plies, `G4 – K8`). **(c)** Group 3, Game 3 (38 plies, `G12 – K8`).

5.3 Analysis of CoT Spatial Occupancy Hallucinations (96 vs. 42)

In Group 3, we tracked every instance of **spatial occupancy hallucinations**—where an agent's CoT reasoning concludes by outputting a coordinate that is already occupied on the board (or exhausts its 4,096 CoT token ceiling while miscounting grid rows):

- **Agent A (2D Matrix Baseline) triggered 96 illegal-move occupancy hallucinations** (consuming `355,743` reasoning tokens), repeatedly losing track of column indices while counting `.` characters across rows in its CoT.
- **Agent B (Three.js 3D/4D Scene Code) reduced spatial hallucinations by 56.3% (42 retries)** while consuming **13.2% fewer reasoning tokens (`308,778` tokens)**. Similarly, in Group 1 (`gemini-3.8-flash-high`), Agent B reduced average per-move reasoning latency by **32.1% (15.2s vs. 22.4s per move)**.

5.4 Complete Game-by-Game Telemetry

Table 2: Game-by-Game Telemetry for Group 3 (`DeepSeek-V4.1-Flash-1017`, Thinking ON, Score: 9–1)

Game	Black (1st)	White (2nd)	Winner	Win Axis	Diagonal?	Plies	Winning 5-in-a-Row Line
DST-G1	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Vertical (|)	No	22	I10 – I9 – I8 – I7 – I6
DST-G2	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Anti-Diag (/)	**Yes (#1)**	31	J5 – I4 – H3 – G2 – F1
DST-G3	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Main-Diag (\)	**Yes (#2)**	38	G12 – H11 – I10 – J9 – K8
DST-G4	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Anti-Diag (/)	**Yes (#3)**	19	L11 – K10 – J9 – I8 – H7
DST-G5	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Vertical (|)	No	18	I10 – I9 – I8 – I7 – I6
DST-G6	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Anti-Diag (/)	**Yes (#4)**	35	K8 – J7 – I6 – H5 – G4
DST-G7	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Anti-Diag (/)	**Yes (#5)**	12	J9 – I8 – H7 – G6 – F5
DST-G8	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Anti-Diag (/)	**Yes (#6)**	35	K10 – J9 – I8 – H7 – G6
DST-G9	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Anti-Diag (/)	**Yes (#7)**	10	L11 – K10 – J9 – I8 – H7
DST-G10	**Agent B (3D)**	Agent A (2D)	Agent A (White)	Vertical (|)	No	22	E11 – E10 – E9 – E8 – E7

Table 3: Game-by-Game Telemetry for Group 2 (`DeepSeek-V4.1-Flash-1017`, Thinking OFF, Score: 10–0) & Group 1 (`gemini-3.8-flash-high`, Score: 4–3–3)

Group & Game	Black (1st)	White (2nd)	Winner	Win Axis	Diagonal?	Plies	Winning Line / Outcome
DS-G1 (No-CoT)	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Main-Diag (\)	**Yes (#8)**	26	G4 – H5 – I6 – J7 – K8
DS-G2, G6, G8, G10	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Vertical (|)	No	11–15	H4 – H5 – H6 – H7 – H8 (4 games)
DS-G3 (No-CoT)	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Vertical (|)	No	14	H3 – H4 – H5 – H6 – H7
DS-G4 (No-CoT)	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Horizontal (−)	No	27	F7 – G7 – H7 – I7 – J7
DS-G5, G7, G9	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Horizontal (−)	No	22–30	Row 9 / Row 11 5-in-a-row (3 games)
GM-G1 (Gemini)	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Anti-Diag (/)	**Yes (#9)**	10	J9 – I8 – H7 – G6 – F5
GM-G2, G4 (Gemini)	**Agent B (3D)**	Agent A (2D)	**Agent B (Black)**	Horizontal (−)	No	29–31	D10 – H10 / C5 – G5
GM-G7 (Gemini)	Agent A (2D)	**Agent B (3D)**	**Agent B (White)**	Vertical (|)	No	24	F10 – F9 – F8 – F7 – F6
GM-G3, G5, G9	Agent A (2D)	**Agent B (3D)**	Agent A (Black)	Horiz / Vert	No	27–37	E9 – I9 / I12 – I8 / I9 – M9
GM-G6, G8, G10	**Agent B (3D)**	Agent A (2D)	Draw (Agreed)	Late-Game Draw	No	28–34	Balanced defensive equilibrium

6. Discussion, Limitations & Future Verification

6.1 Preliminary Theoretical Conjecture: Compute–Representation Equivalence

Why does changing the input format from a 2D ASCII table to Three.js 3D/4D code produce such a pronounced empirical shift? As a **preliminary theoretical conjecture (hypothesis)**, we speculate a phenomenon of **Compute–Representation Equivalence**: an impoverished spatial representation (such as a 1D-serialized 2D ASCII matrix) may force a reasoning model to expend substantial test-time CoT compute merely to mentally reconstruct neighborhood topology that a structured geometric representation (Three.js 3D/4D Scene Code) provides in the prompt. In Group 1 (`gemini-3.8-flash-high`), the win-rate gap narrowed to `4W – 3D – 3L` while consuming +47% extra thinking time per move on the 2D baseline, whereas on `DeepSeek-V4.1-Flash-1017`, the 3D/4D representation dominated in both non-thinking (`10 – 0`) and thinking (`9 – 1`) modes. However, we stress that this remains a working conjecture rather than a proven internal mechanism.

6.2 Study Limitations, Unproven Underlying Mechanisms & Future Verification

Statement of Experimental Scope, Mechanistic Uncertainty & Funding Constraints:
Because this research was conducted independently without institutional funding (using out-of-pocket API expenditures of approximately 3.5M tokens), we must emphasize several important limitations before drawing universal scientific conclusions:

1. **Phenomenon Observed vs. Underlying Cause Unproven:** Crucially, this study currently reports an **empirical phenomenon** rather than a proven internal mechanism. While our explanations regarding 1D token-stride dilution, 4D coordinate invariants, and Three.js code priors represent plausible **preliminary conjectures**, we have not yet isolated the exact causal mechanism inside the neural network (e.g., whether the performance gain primarily stems from the JavaScript/Three.js syntax prior, the 4D scalar invariants, the ray segments, or internal attention-head dynamics). Uncovering the true underlying cause requires fine-grained component ablations and white-box mechanistic probing methods in future research.
2. **Limited Sample Size per Group (N = 10 Games per Group, N = 30 Total):** While the combined 23–4 margin achieves statistical significance (p = 0.00031) due to the large effect size on `DeepSeek-V4.1-Flash-1017` (19–1), the 10-game sample on `gemini-3.8-flash-high` (4W–3D–3L) is too small to establish narrow win-rate differentials with high statistical power.
3. **Limited Model Coverage (Only 2 Models Evaluated):** Due to personal funding constraints, our experiments thus far only evaluate two models from Google and DeepSeek (`gemini-3.8-flash-high` and `DeepSeek-V4.1-Flash-1017`). How other untested models perform under this representation remains to be empirically evaluated.
4. **Future Plan & Research API Compute Grant Application:** Consequently, the findings presented in this paper represent **preliminary empirical results**. To strengthen scientific rigor and investigate the underlying mechanisms, we subsequently plan to apply for **Research API Compute Grants** from major AI research laboratories (such as Google and other research institutions) to continue these experiments across more models, larger game sample sizes, and deeper verification methods.

7. Conclusion

We introduced **Scene-Code-as-Topology**, an exploratory framework demonstrating that compiling discrete 2D grid states into executable Three.js 3D/4D procedural scene graph code substantially mitigates 1D token-stride and diagonal blindness in tested Large Language Models. Across our initial self-funded benchmark of 30 controlled self-play games (719 moves), the 3D/4D representation achieved an 85.2% decisive win rate (23W–3D–4L, p = 0.00031), a 9–0 sweep on diagonal victories, a 56.3% reduction in CoT spatial hallucinations, and a 32.1% reduction in reasoning latency. While larger sample sizes and broader multi-model evaluations are required to establish universal generalizability, these preliminary findings suggest that structured 3D/4D scene code offers a powerful, training-free inductive bias for spatial reasoning. All code, raw JSON telemetry, interactive 3D workbenches, and 1080p match videos are publicly available at `https://github.com/pathetic777/scene-code-as-topology-gomoku`.

References

1. P. Rahmanzadehgervi, L. Bolton, M. R. Taesiri, & A. T. Nguyen (2024). "Vision Language Models are Blind: Failing to Translate Detailed Visual Features into Words," *arXiv preprint arXiv:2407.06581*.
2. X. Wang, B. Zhuang, & Q. Wu (2025). "Are Large Vision Language Models Good Game Players?" *arXiv preprint arXiv:2503.02358*.

3. M. Bai, A. K. Cohen, E. Koss, & C. Lichtenbaum (2025). "Stuck in the Matrix: Probing Spatial Reasoning in Large Language Models," *arXiv preprint arXiv:2510.20198*.
4. W. Li, Y. Zhu, R. Das, & P. Dube (2026). "Do LLMs Build Spatial World Models? Evidence from Grid-World Maze Tasks," *arXiv preprint arXiv:2604.10690*.
5. H. Wang (2025). "LLM-Gomoku: A Large Language Model-Based System for Strategic Gomoku with Self-Play and Reinforcement Learning," *arXiv preprint arXiv:2503.21683*.
6. Y. Yang et al. (2026). "Code-as-Room: Generating 3D Rooms from Top-Down View Images via Agentic Code Synthesis," *arXiv preprint arXiv:2605.18451*.
7. L. C. Her et al. (2025). "Zero-shot 3D Map Generation with LLM Agents: A Dual-Agent Architecture for Procedural Content Generation," *arXiv preprint arXiv:2512.10501*.
8. L. Gao et al. (2023). "PAL: Program-Aided Language Models," *International Conference on Machine Learning (ICML)*.
9. W. Chen et al. (2023). "Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks," *Transactions on Machine Learning Research (TMLR)*.
10. J. Wei et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models," *Advances in Neural Information Processing Systems (NeurIPS)*, 35, 24824–24837.
11. DeepSeek-AI (2025). "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," *arXiv preprint arXiv:2501.12948*.